```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
from sklearn.metrics import (
    confusion_matrix,
    classification_report
)

print("TensorFlow version:", tf.__version__)
print("Libraries imported successfully!")
```

```
TensorFlow version: 2.20.0
Libraries imported successfully!
```

```
# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)

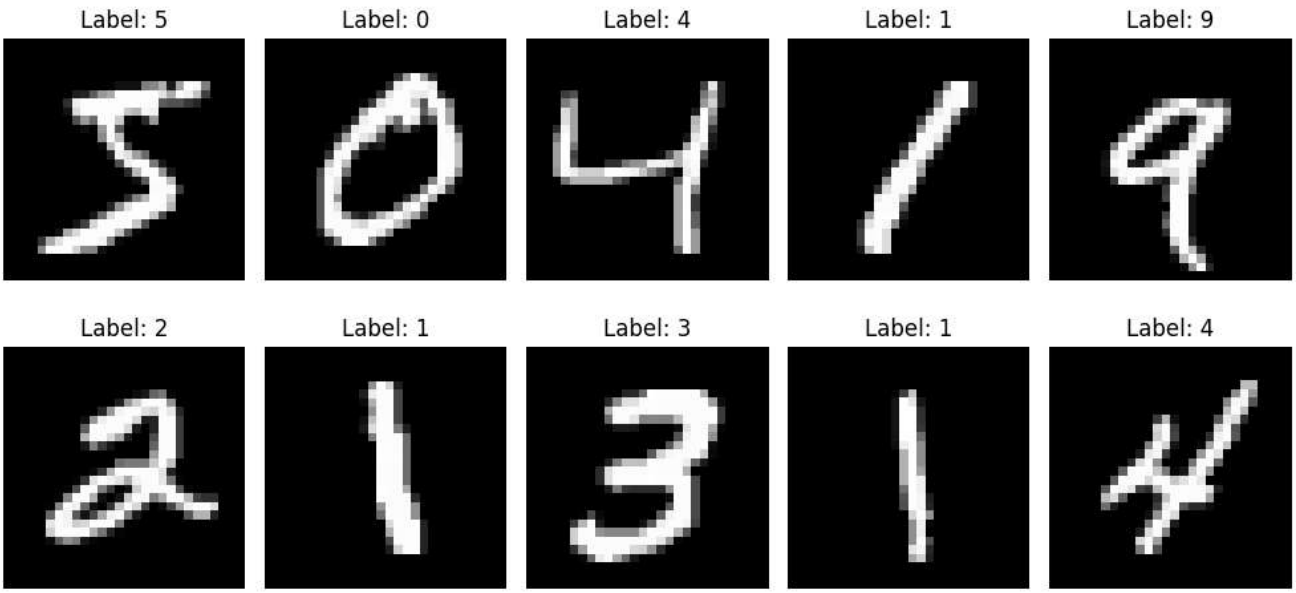
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# Normalize pixel values from 0-255 to 0-1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

```
Training data shape: (60000, 28, 28, 1)
Testing data shape: (10000, 28, 28, 1)
```

```
from sklearn.model_selection import train_test_split

x_train, x_val, y_train, y_val = train_test_split(
    x_train,
    y_train,
    test_size=0.10,
    random_state=42,
    stratify=y_train
)

print("Training data:", x_train.shape)
print("Validation data:", x_val.shape)
print("Testing data:", x_test.shape)
```

```
Training data: (54000, 28, 28, 1)
Validation data: (6000, 28, 28, 1)
Testing data: (10000, 28, 28, 1)
```

```
model = models.Sequential([

    # First convolutional block
    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),
```

```
    layers.MaxPooling2D((2, 2)),

    # Second convolutional block
    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D((2, 2)),

    # Convert feature maps into a vector
    layers.Flatten(),

    # Fully connected layer
    layers.Dense(
        128,
        activation="relu"
    ),

    # Output layer
    layers.Dense(
        10,
        activation="softmax"
    )
])

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

```
history = model.fit(
    x_train,
```

```
    y_train,
    validation_data=(x_val, y_val),
    epochs=10,
    batch_size=64,
    verbose=1
)
```

```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 28s 32ms/step - accuracy: 0.9491 - loss: 0.1693 - val_accuracy: 0.9778 - val_loss: 0.0716
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 27s 32ms/step - accuracy: 0.9846 - loss: 0.0504 - val_accuracy: 0.9863 - val_loss: 0.0542
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 31ms/step - accuracy: 0.9889 - loss: 0.0348 - val_accuracy: 0.9852 - val_loss: 0.0504
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 27s 32ms/step - accuracy: 0.9921 - loss: 0.0257 - val_accuracy: 0.9853 - val_loss: 0.0498
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 27s 32ms/step - accuracy: 0.9941 - loss: 0.0191 - val_accuracy: 0.9868 - val_loss: 0.0517
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 28s 33ms/step - accuracy: 0.9949 - loss: 0.0156 - val_accuracy: 0.9890 - val_loss: 0.0433
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 26s 31ms/step - accuracy: 0.9961 - loss: 0.0122 - val_accuracy: 0.9892 - val_loss: 0.0428
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 31ms/step - accuracy: 0.9964 - loss: 0.0105 - val_accuracy: 0.9828 - val_loss: 0.0716
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 32ms/step - accuracy: 0.9967 - loss: 0.0085 - val_accuracy: 0.9890 - val_loss: 0.0549
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 27s 32ms/step - accuracy: 0.9982 - loss: 0.0053 - val_accuracy: 0.9853 - val_loss: 0.0746
```

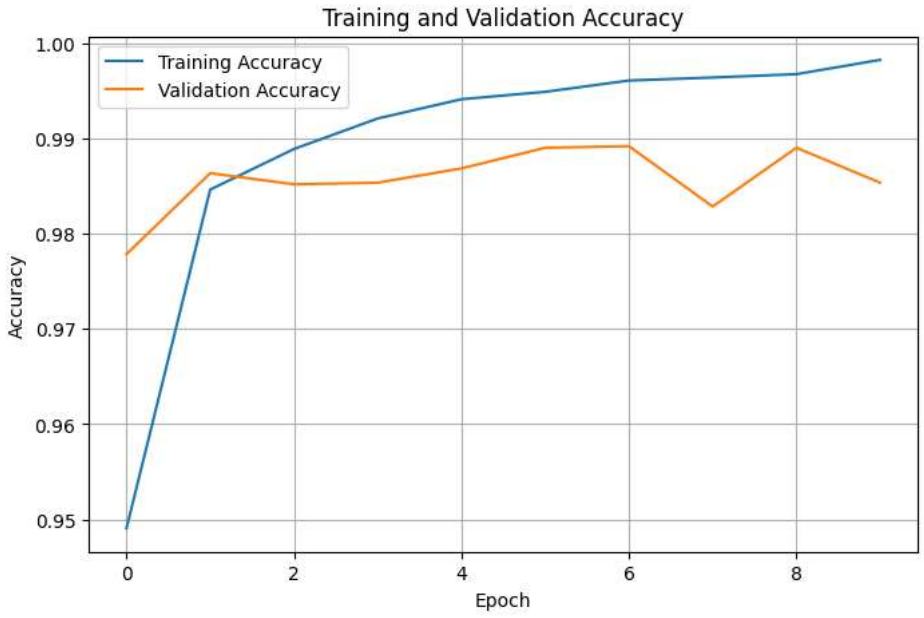
```
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



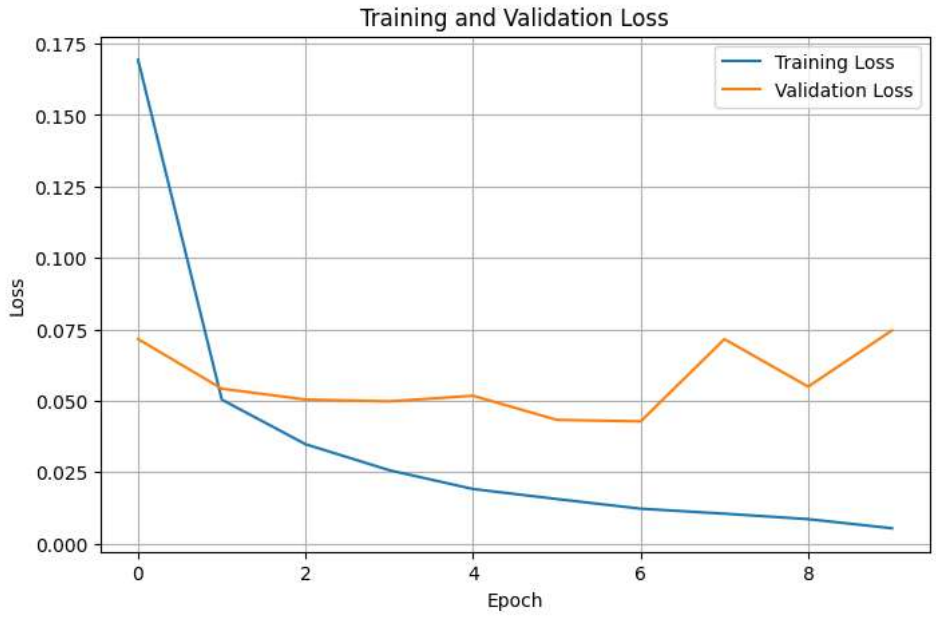

```
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```




```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(f"Test Loss     : {test_loss:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")
print(f"Test Accuracy : {test_accuracy * 100:.2f}%")
```

```
Test Loss     : 0.0516
Test Accuracy : 0.9885
Test Accuracy : 98.85%
```

```
predictions = model.predict(x_test)

predicted_labels = np.argmax(
    predictions,
    axis=1
)

print("Predictions generated successfully!")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
Predictions generated successfully!
```

```
plt.figure(figsize=(12, 6))

for i in range(15):

    plt.subplot(3, 5, i + 1)

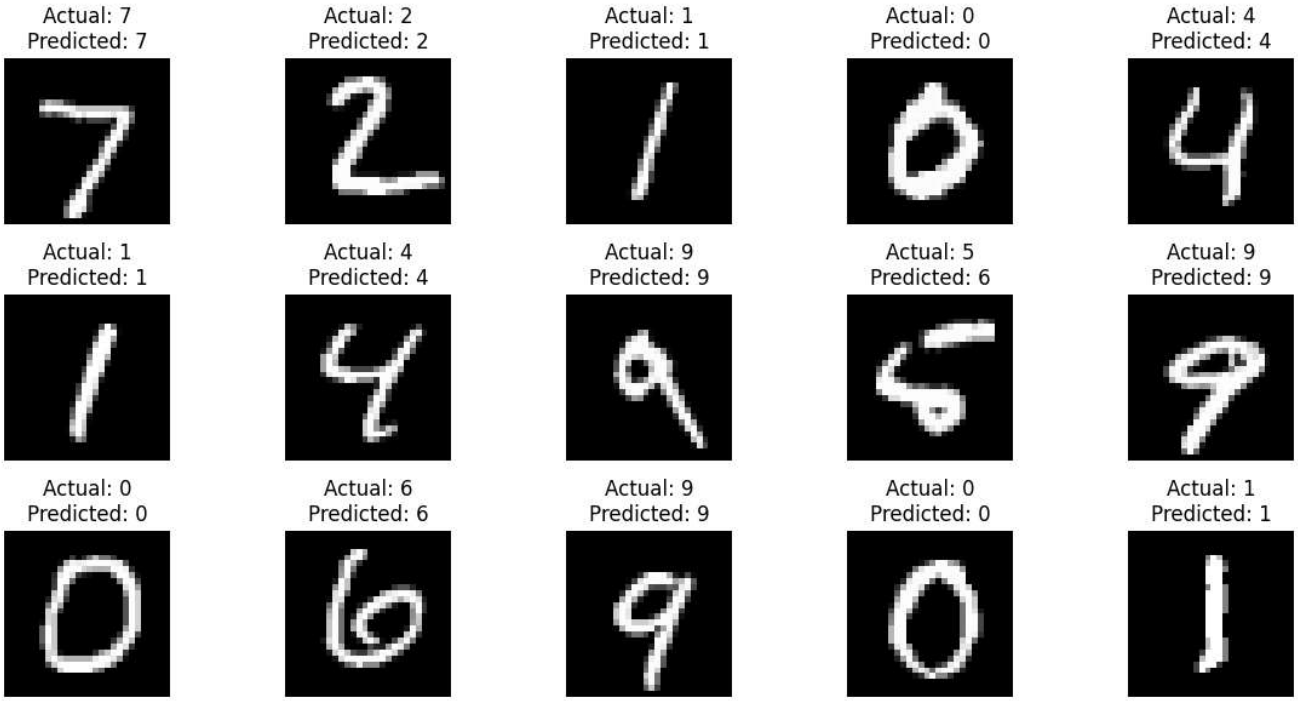
    plt.imshow(
        x_test[i].squeeze(),
        cmap="gray"
    )

    plt.title(
        f"Actual: {y_test[i]}\n"
        f"Predicted: {predicted_labels[i]}"
```

```
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
cm = confusion_matrix(
    y_test,
    predicted_labels
)

plt.figure(figsize=(8, 6))

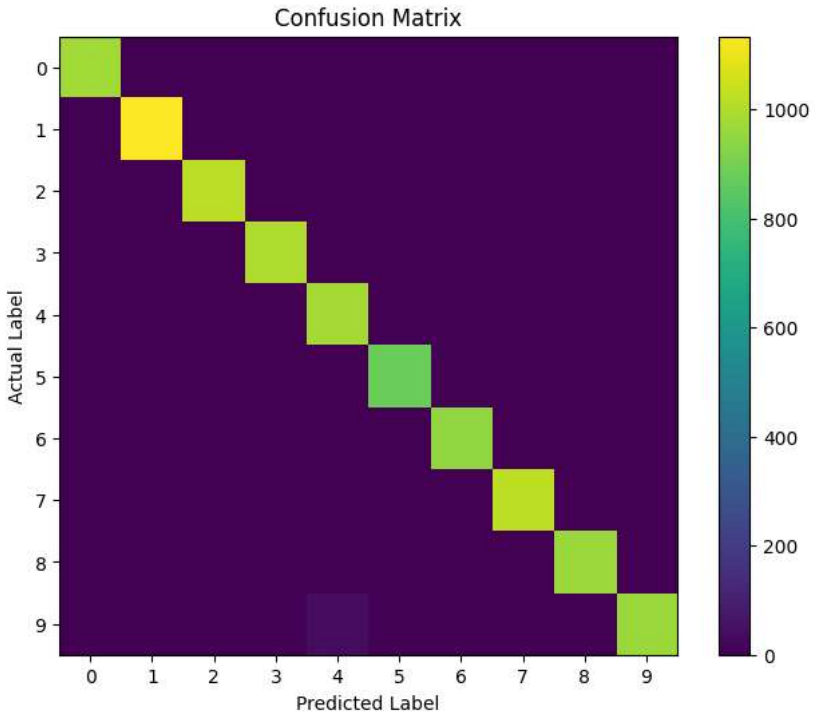
plt.imshow(cm)

plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.colorbar()

plt.xticks(range(10))
plt.yticks(range(10))

plt.show()
```




```
print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[ 976    1    0    0    1    0    1    1    0    0]
 [   0 1133    0    1    0    1    0    0    0    0]
 [   2    1 1018    1    2    0    0    7    1    0]
 [   0    0    0 1000    0    6    0    1    3    0]
 [   0    0    0    0  981    0    0    0    0    1]
 [   2    0    0    5    0  878    5    0    2    0]
 [   2    3    0    0    2    1  950    0    0    0]
 [   0    2    1    0    0    0    0 1023    0    2]
 [   2    1    2    0    0    0    2    2  964    1]
 [   1    1    0    0   39    4    0    2    0  962]]
```

```
print(
    classification_report(
        y_test,
        predicted_labels
    )
)
```

```
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       980
           1       0.99      1.00      1.00      1135
           2       1.00      0.99      0.99      1032
           3       0.99      0.99      0.99      1010
           4       0.96      1.00      0.98       982
           5       0.99      0.98      0.99       892
           6       0.99      0.99      0.99       958
           7       0.99      1.00      0.99      1028
           8       0.99      0.99      0.99       974
           9       1.00      0.95      0.97      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

```
misclassified = np.where(
    predicted_labels != y_test
)[0]
```

```
print(
    "Number of misclassified images:",
    len(misclassified)
)
```

```
Number of misclassified images: 115
```

```
plt.figure(figsize=(12, 8))

for i in range(min(20, len(misclassified))):

    index = misclassified[i]

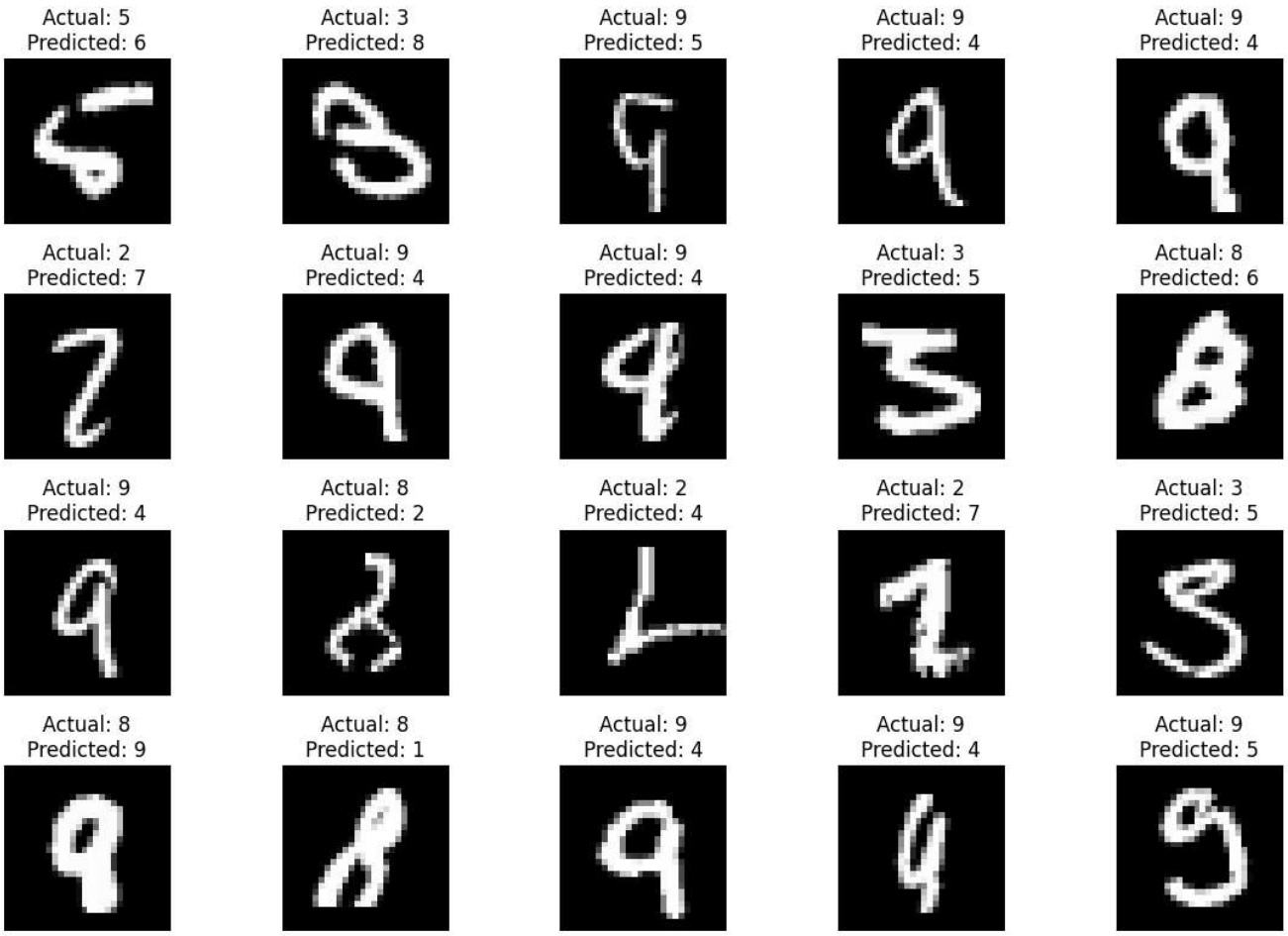
    plt.subplot(4, 5, i + 1)

    plt.imshow(
        x_test[index].squeeze(),
        cmap="gray"
    )

    plt.title(
        f"Actual: {y_test[index]}\n"
        f"Predicted: {predicted_labels[index]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
correct_predictions = np.sum(
    predicted_labels == y_test
)

total_predictions = len(y_test)

manual_accuracy = (
    correct_predictions /
    total_predictions
)

print("Correct Predictions:", correct_predictions)
print("Total Predictions:", total_predictions)
print(
    f"Manual Accuracy: "
    f"{manual_accuracy * 100:.2f}%"
)
```

```
Correct Predictions: 9885
Total Predictions: 10000
Manual Accuracy: 98.85%
```

```
model.save("mnist_cnn_model.keras")

print("Model saved successfully!")
```

```
Model saved successfully!
```

```
from google.colab import files

files.download("mnist_cnn_model.keras")
```

OBSERVATIONS

1. The CNN successfully learned features from the MNIST handwritten digit dataset.
2. Convolutional layers extracted important visual features from the input images.
3. Pooling layers reduced the spatial dimensions of the feature maps.
4. The fully connected layer used the extracted features for classification.
5. The Softmax output layer generated probabilities for the ten digit classes.
6. Training and validation accuracy increased during training, while the corresponding loss generally decreased.
7. The confusion matrix showed that most test images were classified correctly.
8. Some handwritten digits were misclassified because of similarities in their shapes and writing styles.
9. Precision, Recall and F1-Score were calculated using the classification report.
10. The CNN demonstrated the effectiveness of deep learning for handwritten digit classification.

Q1. What is image classification? How does it differ from object detection and image segmentation?

Answer:

Image classification is the process of assigning a class or category label to an entire image.

Object Detection: Object detection identifies objects present in an image and determines their locations using bounding boxes along with class labels.

Image Segmentation: Image segmentation divides an image into different regions or assigns a class label to individual pixels.

Task	Output
Image Classification	Class label
Object Detection	Class label + Bounding box
Image Segmentation	Pixel-level regions

Therefore, image classification identifies what is present in an image, object detection identifies what and where the objects are, while segmentation provides detailed pixel-level information.

Q2. Explain the architecture and working principle of a Convolutional Neural Network (CNN).

Answer:

A Convolutional Neural Network (CNN) is a deep learning model commonly used for image classification. CNNs automatically learn hierarchical features from images.

The basic architecture of a CNN consists of:

Input Image → Convolution Layer → Activation Function → Pooling Layer → Flatten Layer → Fully Connected Layer → Output Layer

1. **Input Layer:** Receives the input image.
2. **Convolution Layer:** Applies filters to the image to extract important features such as edges, shapes, and patterns.
3. **Activation Function:** Usually ReLU is used to introduce non-linearity into the network.
4. **Pooling Layer:** Reduces the spatial dimensions of feature maps while retaining important information.
5. **Flatten Layer:** Converts the feature maps into a one-dimensional vector.
6. **Fully Connected Layer:** Uses the extracted features for classification.
7. **Output Layer:** Produces the final class prediction. Softmax can be used to obtain probabilities for multiple classes.

Thus, CNNs learn useful image features automatically and use these features to classify images.

Q3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?

Answer:

1. Convolutional Layer

The convolutional layer extracts important features from an image using filters or kernels. These features may include edges, textures, shapes, and patterns.

2. Pooling Layer

The pooling layer reduces the spatial dimensions of feature maps. It decreases computational requirements while retaining important features.

Common pooling methods include:

- Max Pooling
- Average Pooling

3. Fully Connected Layer

The fully connected layer receives the extracted features and uses them to perform the final classification.

Therefore, convolutional layers extract features, pooling layers reduce feature-map dimensions, and fully connected layers perform classification.

Q4. Why is image normalization performed before training a deep learning model?

Answer:

Image normalization is performed to scale pixel values into a smaller and more suitable numerical range before training a deep learning model.

For example, an image with pixel values ranging from **0 to 255** can be normalized to a range of **0 to 1** using:

Normalized Pixel = Pixel Value / 255

Normalization helps make the training process more stable and allows the neural network to learn efficiently.

In the MNIST dataset, normalization is performed by dividing each pixel value by 255.

Q5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs.

Answer:

ReLU

ReLU stands for **Rectified Linear Unit**.

It is defined as:

ReLU(x) = max(0, x)

ReLU converts negative values to zero while keeping positive values. It introduces non-linearity into the CNN and helps the network learn complex patterns.

Softmax

Softmax is generally used in the output layer for multi-class classification.

It converts the output values of the network into probabilities for different classes.

For example, in the MNIST dataset, the output layer contains 10 classes corresponding to digits **0 to 9**.

The class having the highest probability is selected as the predicted class.

Q6. What is a Confusion Matrix? How is it used to evaluate classification performance?

Answer:

A **Confusion Matrix** is a table used to evaluate the performance of a classification model.

It compares the actual class labels with the predicted class labels.

For a multi-class classification problem:

- **Rows** represent actual classes.
- **Columns** represent predicted classes.
- **Diagonal values** represent correctly classified samples.
- **Off-diagonal values** represent incorrectly classified samples.

A confusion matrix helps identify which classes are being classified correctly and which classes are being confused with one another.

For example, in MNIST, it can show whether the model is confusing one handwritten digit with another.

Therefore, a confusion matrix provides detailed information about the classification performance of a model.

Q7. Differentiate between Accuracy, Precision, Recall, and F1-Score with suitable examples.

Answer:

Accuracy

Accuracy represents the proportion of total predictions that are correctly classified.

Formula:

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision

Precision represents how many of the samples predicted as positive are actually positive.

Formula:

Precision = TP / (TP + FP)

Recall

Recall represents how many of the actual positive samples were correctly identified.

Formula: